FlyWire Cell-Type Prediction — Project Plan

Oct 18, 2026 · @Don

Summary

You will predict each FlyWire neuron's cell type from its wiring, benchmark your models against a published method, and ship an explorer that shows where and why predictions go wrong. Everything runs on a laptop CPU; no GPU is needed at any step.

Why it stands out: a real 139,255-node graph with a long-tailed label set (thousands of types), a published baseline to measure yourself against, and a public tool people can click through. Very few student portfolios have a dataset like this.

Two traps to design around from day 1

- **This has been done well already.** NTAC, a simple connectivity-only method, reports over 95% accuracy on the fly's visual system with 2% of neurons labeled, in minutes on a normal PC. Don't pitch "first to predict cell types". Pitch a rigorous comparison, plus a question the NTAC authors themselves leave open: does adding non-connectivity features (where a neuron's synapses sit, its predicted neurotransmitter) help, especially in the central brain where accuracy is lowest?
- **Some labels were made from connectivity.** Optic-lobe types were defined largely by connectivity, so high accuracy there partly recovers the labeling procedure. Report brain regions separately and say this plainly in the README.

Data

Use release 783, which is still FlyWire's latest public whole-brain release, and pull it from the two archived sources below rather than Codex, so your numbers are reproducible.

File	Source	Size	What you use it for
`proofread_connections_783.feather`	Zenodo	852 MB	Edges: one row per neuron pair per brain region, with synapse count and average neurotransmitter probabilities

File	Source	Size	What you use it for
`per_neuron_neuropil_count_pre_783.feather` / `_post_`	Zenodo	17 MB / 234 MB	Where each neuron's output and input synapses sit
`proofread_root_ids_783.npy`	Zenodo	1 MB	The 139,255 proofread neuron IDs
`Supplemental_file1_neuron_annotations.tsv`	flywire_annotations, tag v3.1.0	small	Labels: super class, class, cell type, side, plus columns you must never use as features

Skip `flywire_synapses_783.feather` (9.5 GB); you don't need individual synapse points.

Gotchas

- **Aggregate before thresholding.** The edge file has one row per pair *per brain region*. Sum across regions first, then keep pairs with 5+ synapses, the same default Codex uses.
- **Check every download** against the md5 checksums on the Zenodo page, and record them in the repo.
- **Pin the annotation tag.** Annotations are still being revised (v3.0.0 and v3.1.0 changed cell types), and Codex mixes sources, so its labels can differ from the GitHub ones.
- **Left/right was flipped in the original imaging.** Annotations were corrected but images were not; trust the `side` column, never coordinates, for anything hemisphere-related.
- **Licence and credit:** the connectivity data is CC-BY 4.0. Cite Dorkenwald et al. 2024, Schlegel et al. 2024, Matsliah et al. 2024 and Berg et al. 2025, as the annotation repo asks, in the README and in the explorer's footer.

Labels form a hierarchy: super class (9 values, e.g. optic, central, sensory), class, then cell type. The annotation paper describes 8,453 types, most with very few neurons each.

Task and evaluation

The setting is semi-supervised: some neurons are labeled ("seeds") and the model labels the rest, the way a lab would type a newly reconstructed brain. How you split and what you let the model see decide whether your numbers mean anything.

What to predict, in increasing difficulty: super class, class, then cell type. Get the first two working end to end before touching the 8,000-way problem.

Three evaluation settings

1. **Seed fraction sweep:** label 1%, 5%, 20% and 80% of neurons at random, guaranteeing at least one seed per type. This matches NTAC's protocol, so your numbers sit next to theirs.
2. **Hemisphere transfer:** labels on one side only, predict the other. Closest to real use, and the hardest for rare types that have one neuron per side.
3. **By region:** report optic lobe and central brain separately in both settings.

Leakage rules

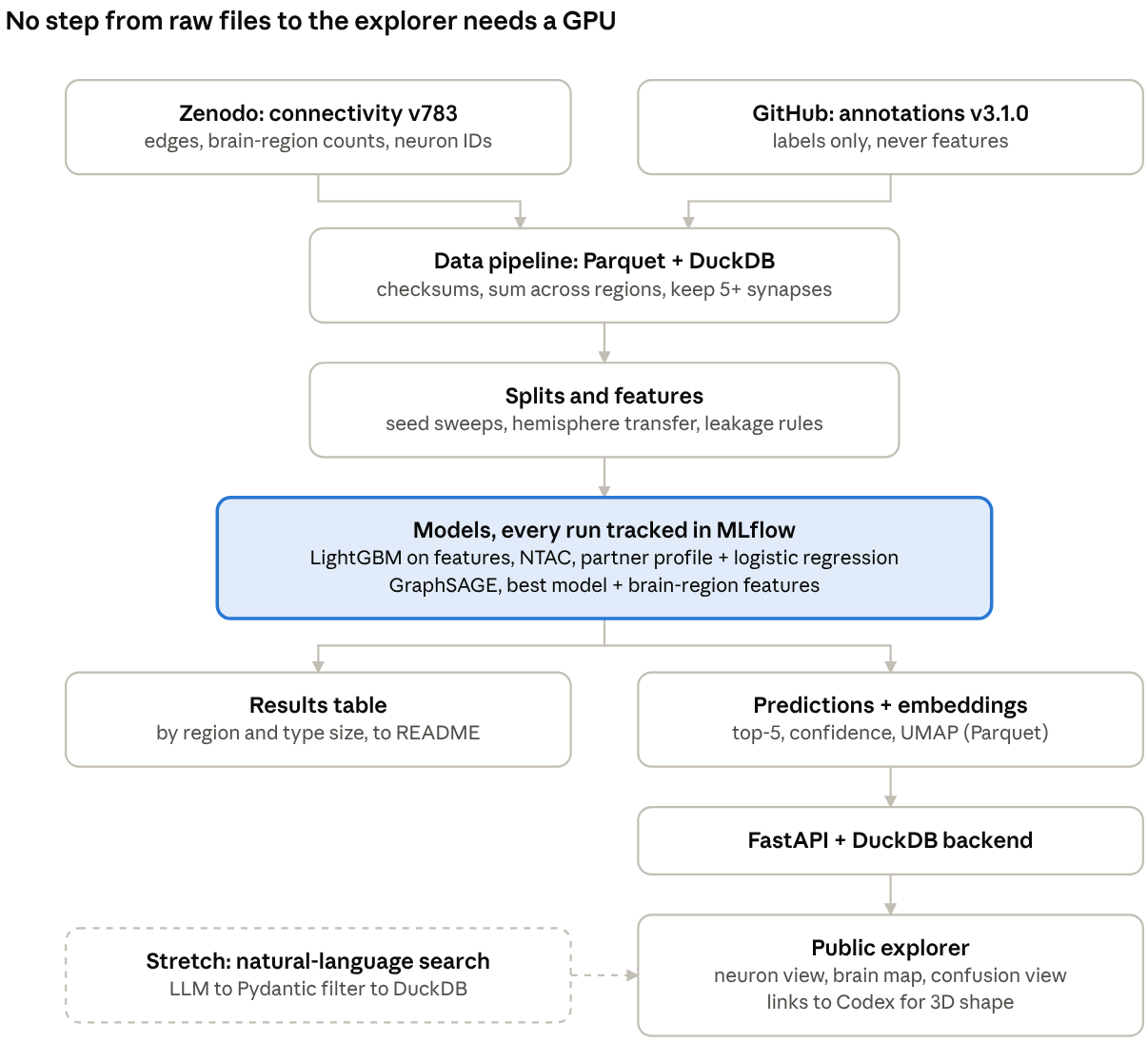
- Partner-type features may only use seed labels, never test labels.
- Never use annotation columns as features: `hemibrain_type`, mirror-twin IDs, synonyms, lineage or anything derived from `cell_type`.
- Tune on a validation split; touch the test split once per final model.
- Fix and log random seeds; every number in the README is a mean over 5 runs, as in NTAC.

Metrics: accuracy and weighted F1 (to compare with NTAC), top-5 accuracy (useful to a human labeler), macro F1 (rare types count equally), accuracy by type size (2, 3–10, 11–100, over 100 neurons), and calibration, because the explorer shows confidence.

Method	Uses	Why it's in the lineup
Majority class	Nothing	Floor
LightGBM on neuron features	Degree, synapse totals, synapses per brain region, neurotransmitter probabilities	How far "where and what" goes with no partner labels
NTAC, seeded	Synapse counts to and from each seed type, Jaccard nearest seed	The published reference; implement it yourself and check against their code
Partner-type profile + logistic regression	NTAC's features fed to a learned classifier	Is a trained model better than nearest-seed?
GraphSAGE	Neuron features plus message passing over weighted, directed edges	The graph neural network

Method	Uses	Why it's in the lineup
Best model + neuron features	Partner profile plus brain-region and neurotransmitter features	Your open question: do extra features help in the central brain?

Architecture



pipeline · raw files to models to explorer

Labels enter only as targets and seed labels; the explorer reads precomputed Parquet, so the server never trains or scores anything live.

Week 1: Data pipeline and exploration

By the end of the week, one command rebuilds a clean graph from the raw files, and a notebook shows what the data looks like.

Must-have

- [ ] `make data`: download the four files, verify checksums, aggregate edges across brain regions, keep pairs with 5+ synapses, write Parquet.
- [ ] Load into DuckDB; join labels; one row per neuron with its features (degrees, synapse totals, synapses per brain region, mean neurotransmitter probabilities).
- [ ] Sanity checks against published numbers: 139,255 neurons, and about 2.7 million edges after thresholding. Investigate any big gap before moving on.
- [ ] Exploration notebook: neurons per super class, type-size distribution (expect a long tail), degree distributions, reciprocal connections, and how many neurons lack a type.
- [ ] A data card in the README: sources, versions, checksums, licence, citations.

Stretch

- [ ] Do same-type neurons connect to each other, or just to the same kinds of partners? Measure it. The answer shapes Week 3.

Done when: a fresh clone plus `make data` reproduces the same row counts.

Week 2: Splits and baselines

By the end of the week you have reproduced one NTAC result and filled the first rows of the results table.

Must-have

- [ ] Split code for all three evaluation settings, with saved seed lists so every model sees identical splits.
- [ ] MLflow tracking for every run: method, split, seed fraction, region, metrics, git commit.
- [ ] Majority and LightGBM-on-features baselines.
- [ ] Your own seeded NTAC: build each neuron's synapse counts to and from every seed type, assign the nearest seed by Jaccard distance, repeat. Keep the count matrix sparse (SciPy) or it won't fit in memory.
- [ ] Reproduce one NTAC setting first, such as the visual system at a low seed fraction, and land within a few points of their reported figure. If you can't, find out why before going further.

- [ ] Partner-type profile + logistic regression.
- [ ] ADR-001: split design and leakage rules.

Stretch

- [ ] Run their package on the same splits and confirm your implementation matches it.

Done when: the results table has every baseline for super class and class, plus cell type in at least one region.

Week 3: Graph neural network on CPU

By the end of the week you know whether a GNN beats NTAC, by how much, and in which regions. "It doesn't" is a valid, useful answer.

Must-have

- [ ] PyTorch Geometric GraphSAGE with separate aggregation for incoming and outgoing edges, and edge weight log(1 + synapses). Two layers, full-batch on CPU; the graph fits comfortably in laptop memory.
- [ ] Node features from Week 1; seed labels fed in as an extra input for seeds only (the "label as input" trick), never for test neurons.
- [ ] Ablations: connectivity only, + brain-region features, + neurotransmitter features.
- [ ] Your open question: add brain-region and neurotransmitter features to the best Week 2 model and check whether central-brain accuracy improves.
- [ ] All runs logged; results table complete for every method × setting × region.
- [ ] ADR-002: model choice and what the ablations showed.

Stretch

- [ ] Graph attention network (GAT) for comparison.
- [ ] Free Colab or Kaggle GPU for larger sweeps. Training is a batch job, so a notebook is fine here.

Why a GNN might lose: message passing averages a neuron's neighbours, but same-type neurons often share the *kinds* of partners they have rather than connecting to each other. NTAC's features capture exactly that. Your Week 1 stretch measurement tells you how much this matters here.

Done when: the results table is final and you can explain every gap in it.

Week 4: Interactive graph explorer

By the end of the week a public link lets anyone search a neuron, see its wiring and the model's prediction, and find where the model goes wrong. It doesn't try to draw the whole brain: 2.7 million edges won't render in a browser.

Three views

1. **Neuron view:** search by ID or type. Shows the top input and output partners as a small graph, coloured by predicted type, with wrong predictions outlined. Side panel: true type, top-5 predictions with confidence, and the partner types that drove the prediction. Link out to the neuron's Codex page for its 3D shape rather than rebuilding that.
2. **Brain map:** a 2D layout (UMAP) of every neuron's learned embedding, coloured by super class. Click a point to open its neuron view.
3. **Confusion view:** the type pairs the model mixes up most, with accuracy by region and type size.

Must-have

- [ ] Precompute predictions, top-5, confidence and the UMAP coordinates into Parquet.
- [ ] FastAPI + DuckDB backend: endpoints for search, neighbourhood (top-k partners) and confusion pairs.
- [ ] Frontend: Sigma.js for the neuron graph, a WebGL scatter (deck.gl) for the brain map.
- [ ] Deploy on a free host such as Hugging Face Spaces (check current limits), with data citations in the footer.

Stretch

- [ ] Compare two models side by side on the same neuron.

Done when: a friend can open the link on their phone, find a neuron and understand why it was predicted wrong.

Week 5: Error analysis and write-up

By the end of the week the README tells a clear story with honest numbers, and the explorer backs it up.

Must-have

- [ ] Error analysis: which type pairs get confused and whether they are sister types; accuracy by type size and region; whether confidence is calibrated.
- [ ] README: one-paragraph pitch, architecture diagram, results table, two or three charts, link to the explorer, limitations (including the labels-from-connectivity caveat), full citations.
- [ ] `make reproduce` runs data → splits → all models → results table.
- [ ] A short write-up post explaining the open-question result in plain words.

Stretch: the AI-engineering layer

- [ ] Natural-language search in the explorer, such as "GABA neurons in the mushroom body predicted as Kenyon cells". An LLM turns the question into a structured filter, Pydantic validates it, DuckDB runs it. Score it on 50 hand-written questions with known answers.

Stretch: a second brain

- [ ] Train on FlyWire, then label neurons in another fly connectome (the male CNS, or the brain-and-nerve-cord dataset BANC) where type names match. This is the real use case: typing a brain nobody has labeled yet.

Done when: someone who has never heard of a connectome can read the README in five minutes and know what you found.

Resume bullets

Fill every blank from the results table, or cut the bullet.

- Built a cell-type classifier for the 139,255-neuron FlyWire fruit-fly connectome, comparing feature models, a graph neural network and the published NTAC method across ___ evaluation settings.
- Reached ___% top-1 and ___% top-5 accuracy on held-out central-brain neurons with ___% of labels; adding brain-region features changed accuracy by ___ points over connectivity alone.
- Designed leakage-safe splits for semi-supervised graph learning (seed sweeps, hemisphere transfer) and reproduced a published result within ___ points.
- Built and deployed an interactive explorer (FastAPI, DuckDB, Sigma.js) for browsing predictions, errors and neuron neighbourhoods across ___ cell types.

Interview talking points

Interviewers at product companies won't care about flies. They care that you handled a hard graph problem carefully, so frame each point that way.

1. **It's the same problem as fraud and account typing.** Few labeled nodes, a large directed graph, label the rest. Fraud rings, bot detection and merchant categorization in payment graphs all look like this.
2. **Leakage in graph ML.** Partner labels, mirror twins and annotation columns can all leak the answer. Say exactly which ones you blocked and how you know.
3. **Why a simple method can beat a GNN.** Message passing smooths over neighbours; here, same-type neurons tend to share partner *kinds* rather than linking to each other. Your Week 1 measurement is the evidence.
4. **Metrics for long tails.** With thousands of types, accuracy hides rare-type failure; macro F1 and accuracy-by-type-size expose it.
5. **Know where your labels came from.** Optic-lobe types were partly defined by connectivity, so high accuracy there is partly circular. Spotting that is a data-scientist instinct.
6. **Engineering at scale on a laptop.** Parquet and DuckDB instead of pandas everywhere, sparse matrices for 8,000-way profiles, neighbourhood views instead of drawing 2.7 million edges.

Risks and cuts

Alongside classes, plan on 5 to 6 weeks; the week labels are an order, not a deadline.

Risk	How you'll notice	Response
Laptop runs out of memory	Week 1 load or Week 2 profiles	Keep matrices sparse, work on one region first (optic lobe or central brain), use free Colab or Kaggle for the big runs
Can't reproduce NTAC's number	Week 2 check	Check thresholding, label version and seed sampling against their protocol before building on it
GNN doesn't beat NTAC	Week 3 table	Report it; the ablation and the explorer still carry the project

Risk	How you'll notice	Response
Label version drift	Counts differ from the papers	Pin tag v3.1.0 and print it in every results table
Free host sleeps or is too small	Week 4 deploy	Shrink what the server loads, and keep a recorded demo in the README
Falling behind	Week 3 not done by its end	Cut in this order: second-brain stretch, natural-language search, GAT, confusion view, brain map. Keep the neuron view

Never cut the leakage rules, the NTAC comparison or the by-region reporting. Those are what make the results believable.

Sources

- FlyWire whole-brain connectome connectivity data, v783, Zenodo (CC-BY 4.0)
- flywire_annotations, GitHub: annotation files, version history, citation list
- Codex FAQ: releases, thresholds, the left/right inversion note
- Neuronal wiring diagram of an adult brain, Dorkenwald et al., Nature 2024
- Whole-brain annotation and multi-connectome cell typing of Drosophila, Schlegel et al., Nature 2024
- Connectivity is all you need: inferring neuronal types with NTAC, preprint, and NTAC code
- Neuropil projectome study, arXiv: source for the ~2.7 million thresholded connections